

Day 60 -- Line-by-Line Syntax Explanation

day60_gradient_clipping.py, segment by segment | MD Mutasim Billah

1. The Clipper

the global norm, across arrays of different shapes

IMPLEMENTATION

```
norm = np.sqrt(sum(np.sum(g ** 2) for g in grads.values()))
```

`grads.values()` yields five arrays of different shapes. `np.sum(g ** 2)` collapses each to one number; the outer `sum()` adds the five; `np.sqrt` turns the total into a length. The result equals the norm of all entries laid end to end.

returning the originals for an exact no-op

IMPLEMENTATION

```
if max_norm is None or norm <= max_norm:  
    return grads, norm, 1.0
```

Returning the SAME dict -- not a scaled copy with scale 1.0 -- guarantees `clip=None` reproduces Day 59 bit for bit. The check (`clipped[k] is grads[k]`) confirms it: 'is' tests object identity, not just equal values.

one scale factor for every array

IMPLEMENTATION

```
scale = max_norm / norm  
return {k: g * scale for k, g in grads.items()}, norm, scale
```

A dict comprehension builds a new dict with every array multiplied by the same scalar. Because the factor is shared, every ratio between entries is preserved -- that is what keeps the direction fixed.

value clipping, for contrast

IMPLEMENTATION

```
{k: np.clip(g, -limit, limit) for k, g in grads.items()}
```

`np.clip` replaces any entry above limit with limit and below -limit with -limit, independently per entry. Entries already inside the range are untouched, so ratios between large and small entries change.

2. Using It in Training

clip between computing and applying

IMPLEMENTATION

```
grads, self.pre_clip_norm, self.scale = clip_by_global_norm(grads, self.clip)
self.update_norm = self.lr * self.pre_clip_norm * self.scale
for name, g in grads.items():
    getattr(self, name)[...] -= self.lr * g
```

Unlike Day 58, `backward()` now collects gradients in a dict first, so they can be clipped as a set before any parameter changes. `getattr(self, name)` fetches `self.Wxh` etc. by name; `[...] -=` updates the array in place.

the direction check

IMPLEMENTATION

```
cos = g0 @ g1 / (np.linalg.norm(g0) * np.linalg.norm(g1))
```

The dot product of two vectors divided by the product of their lengths is the cosine of the angle between them: 1 means identical direction regardless of length, lower means the direction has turned.

3. Finding the Collapse

first evaluation after the peak that fell below 0.6

IMPLEMENTATION

```
peak_i = int(np.argmax(eu["test"]))
crash = next(i for i in range(peak_i, len(eu["test"]))) if eu["test"][i] < 0.6
```

`np.argmax` returns the index of the best evaluation. `next()` on a generator returns the first index after it that satisfies the condition -- a compact 'find first' without writing a loop.

the largest gradient inside the collapse window

IMPLEMENTATION

```
lo, hi = eu["step"][crash - 1], eu["step"][crash]
biggest = lo + int(np.argmax(su["pre"][lo:hi]))
```

Evaluations happen every 100 steps, so the collapse lies between two of them. Slicing the per-step gradient record to `[lo:hi]` and taking `argmax` finds the culprit; adding `lo` converts the slice index back to a step number.